

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №6.2

«Быки и коровы»

Выполнил:

студент группы 3825Б1ФИип2
Беспалов Р.т

Проверил:

Ивлев А.Д.

Содержание

Таблица 1. Введение	3
3.1. Описание структуры программы	6
3.2. Описание структур данных	6
3.3. Описание алгоритмов	6
Таблица 2. Заключение	8
Таблица 3. Литература	9
Таблица 4. Страуструп Бьерн Язык программирования C++. Специальное издание. Пер. с англ. – М.: Издательство Бином, 2017 г. – 1136 с.	9
Таблица 5. Сайт cppreference: https://en.cppreference.com/	9
Таблица 6. Приложение	10

Введение

Игра «Быки и коровы» — классическая логическая игра, известная с XIX века. Цель игры — угадать задуманное компьютером число с неповторяющимися цифрами. На каждом ходу игрок получает подсказку в виде количества «быков» (цифр, стоящих на своих местах) и «коров» (цифр, присутствующих в числе, но не на своих местах). Данная лабораторная работа посвящена реализации этой игры на языке C++ с использованием объектно-ориентированного подхода.

1. Постановка задачи

Разработать систему классов и реализовать с ее помощью игру «Быки и коровы».

- Игроют два игрока (человек и компьютер).
- Игрок выбирает длину загадываемого числа — n .
- Компьютер «задумывает» n -значное число с неповторяющимися цифрами.
- Игрок делает попытку отгадать число — вводит n -значное число с неповторяющимися цифрами.

Компьютер сообщает, сколько цифр угадано без совпадения с их позициями в загаданном числе (то есть количество коров) и сколько угадано вплоть до позиции в

загаданном числе (то есть количество быков).

- Игрок делает попытки, пока не отгадает всю последовательность.

Пример.

- Пусть $n = 4$ и задумано тайное число «3219».
- Игрок ввел число «2310».
- Результат: две «коровы» (две цифры: «2» и «3» — угаданы на неверных позициях) и один «бык» (одна цифра «1» угадана вплоть до позиции).

2. Руководство пользователя

Для запуска программы необходимо скомпилировать файл Lab?6.2.cpp любым C++ компилятором (поддерживающим стандарт C++11 и выше). Запуск осуществляется через командную строку или среду разработки.

Порядок работы с программой:

- 1 При запуске программа запрашивает длину загадываемого числа.
- 2 Пользователь вводит целое число от 1 до 10.
- 3 Компьютер генерирует случайное число указанной длины без повторяющихся цифр.
- 4 Программа выводит сообщение Enter your guess:.
- 5 Пользователь вводит число той же длины без повторяющихся цифр.
- 6 Программа выводит количество быков и коров или сообщение об ошибке.
- 7 Шаги 4-6 повторяются до полного отгадывания числа.
- 8 При победе программа выводит загаданное число и количество попыток, после чего завершается.

Возможные сообщения об ошибках:

- Error: Incorrect number length. — длина вне диапазона 1-10.(конец программы)
- Error: Guess size does not match number size. — длина ввода не совпадает с загаданным числом.(повторная попытка)
- Error: Incorrect characters in a guess. Use digits 0-9. — ввод содержит нецифровые символы.(повторная попытка)
- Error: Guess has repeating digits. — ввод содержит повторяющиеся цифры.(повторная попытка)

3. Руководство программиста

3.1. Описание структуры программы

Программа реализована в одном файле Lab?6.2.cpp и состоит из следующих модулей:

- 1 Подключение заголовочных файлов - `<iostream>`, `<string>`, `<cstdlib>`, `<ctime>`, `<stdexcept>`
- 2 Класс `BullsAndCows` - Содержит всю логику игры: генерацию числа, проверку ввода, подсчёт быков и коров
- 3 Функция `main()` - Управляет игровым процессом, обрабатывает исключения и организует цикл ввода

3.2. Описание структур данных

Класс `BullsAndCows` содержит:

- **Приватные поля:** `number` (загаданное число), `attempts` (счётчик попыток)
- **Приватные методы:** `generate()` (генерация числа), `hasDuplicates()` (проверка повторов), `printBullsAndCows()` (подсчёт и вывод)
- **Публичные методы:** конструктор `BullsAndCows(size_t)`, метод `guess(const string&)` (попытка угадать число)

3.3. Описание алгоритмов

Алгоритм генерации числа (`generate(size)`):

- 1 Создаётся строка `digits = "0123456789"`.
- 2 Для каждой позиции от 0 до `size-1`:
 - Вычисляется диапазон: если позиция первая, то от 1 до 9 (исключая ноль), иначе от 0 до текущей длины `digits`.
 - Генерируется случайный индекс `digit` в этом диапазоне.
 - Выбранная цифра `digits[digit]` добавляется в `number` и удаляется из `digits`.

Алгоритм проверки дубликатов (`hasDuplicates(input)`):

- Двойным циклом сравниваются все пары символов во входной строке.
- При нахождении любых двух одинаковых символов возвращается `true`.

Алгоритм подсчёта быков и коров (`printBullsAndCows(input)`):

- 1 Циклом сравниваются пары символов на одной позиции в загаданном и введённом числе. Увеличивается на число совпадения счётчик быков.
- 2 Двойным иклом сравниваются пары символов на разных позициях в загаданном и введённом числе. Увеличивается на число совпадения счётчик коров.

- 3 Если число быков совпадает с длиной загаданного слова(то есть всё верно угадано) выводится сообщение о победе с числом попыток. Возвращается true.
- 4 Иначе выводится число быков и коров. Возвращается false.

Алгоритмы попытки(guess(input)):

- 1 Проверяется корректность ввода на символы(цифры 0-9), отсутствие повторений и длину числа и пишутся сообщения о соответствующих ошибках.
- 2 Если всё корректно увеличивается счётчик попыток на 1 и возвращается printBullsAndCows

Алгоритм игрового цикла в main():

- 1 Ввод длины, создание объекта BullsAndCows.
- 2 Бесконечный цикл:
 - Ввод строки guess.
 - Вызов game.guess(guess).
 - Если возвращено true — выход из программы.
- 1 Перехват и обработка исключения std::out_of_range.

4. Результаты экспериментов и тестирования

- **Тест 1. Корректный ввод, успешное отгадывание**

Сценарий:

- 1 Пользователь вводит длину числа: 4.
- 2 Компьютер генерирует число (например, 3219).
- 3 Пользователь последовательно вводит:
 - 1234 ? ответ: быки: 1, коровы: 2;
 - 3201 ? ответ: быки: 2, коровы: 1;
 - 3219 ? ответ: победа, 3 попытки.

Результат: программа корректно генерирует число, подсчитывает быков и коров, завершает игру с сообщением о победе и указанием количества попыток.

Вывод: алгоритм генерации и подсчёта работает корректно.

- **Тест 2. Ввод длины числа вне допустимого диапазона**

Сценарий: пользователь вводит длину числа 0 или 11.

Ожидаемый результат: программа выводит ошибку `Error: Incorrect number length.` и завершается.

Фактический результат: программа выбрасывает исключение `std::out_of_range`, перехватывает его в `main()` и выводит сообщение об ошибке. Завершается с кодом 1.

Вывод: проверка длины числа реализована корректно, критические ошибки обрабатываются через исключения.

- **Тест 3. Ввод числа неправильной длины**

Сценарий: длина загаданного числа — 4, пользователь вводит 123 (3 цифры) или 12345 (5 цифр).

Ожидаемый результат: сообщение `Error: Guess size does not match number size. Try again..`

Фактический результат: программа выводит указанное сообщение, не увеличивает счётчик попыток и запрашивает новый ввод.

Вывод: проверка длины ввода работает корректно.

- **Тест 4. Ввод нечисловых символов**

Сценарий: длина числа — 4, пользователь вводит 12a3 или abcd.

Ожидаемый результат: сообщение `Error: Incorrect characters in a guess. Use digits 0-9. Try again..`

Фактический результат: программа обнаруживает нечисловые символы, выводит сообщение об ошибке, не увеличивает счётчик попыток.

Вывод: проверка на корректность символов реализована корректно.

- **Тест 5. Ввод повторяющихся цифр**

Сценарий: длина числа — 4, пользователь вводит 1123 или 4444.

Ожидаемый результат: сообщение Error: Guess has repeating digits. Try again..

Фактический результат: программа выявляет повторяющиеся цифры с помощью метода hasDuplicates(), выводит сообщение об ошибке, не увеличивает счётчик попыток.

Вывод: алгоритм проверки дубликатов работает корректно.

- **Тест 6. Крайние случаи длины числа**

Сценарий 6.1: пользователь вводит длину 1. Компьютер генерирует цифру от 1 до 9. Пользователь вводит эту цифру.

Результат: победа с 1 попыткой.

Сценарий 6.2: пользователь вводит длину 10. Компьютер генерирует 10-значное число без повторяющихся цифр (использует все цифры от 0 до 9, первая цифра не 0). Пользователь вводит это число.

Результат: победа с 1 попыткой.

Вывод: программа корректно обрабатывает крайние случаи длины числа (1 и 10).

- **Тест 7. Проверка случайности генерации**

Сценарий: запустить программу 10 раз с длиной числа 4. Записать сгенерированные числа.

Ожидаемые результаты:

- все числа имеют длину 4;
- в каждом числе нет повторяющихся цифр;
- первая цифра не равна 0;
- числа различны в большинстве случаев (из-за случайности).

Фактические результаты: во всех 10 запусках сгенерированные числа соответствуют требованиям: нет повторов, первая цифра от 1 до 9, числа различны.

Вывод: алгоритм генерации числа обеспечивает случайность и соблюдение условий задачи.

Заключение

В ходе выполнения лабораторной работы была разработана и реализована игра «Быки и коровы» на языке C++. Достигнуты следующие результаты:

- 1 Спроектирован и реализован класс BullsAndCows, инкапсулирующий всю логику игры.
- 2 Реализована генерация числа заданной длины без повторяющихся цифр.
- 3 Реализован алгоритм подсчёта быков и коров.
- 4 Реализована полная обработка ошибок ввода (неверная длина, нецифровые символы, повторяющиеся цифры).
- 5 Программа ведёт подсчёт количества попыток и завершается с сообщением о победе.
- 6 Код соответствует стандарту C++11 и выше, не содержит глобальных переменных, использует исключения для обработки критических ошибок.

Литература

Список источников, используемых при выполнении работы (если есть).

Страуструп Бьерн Язык программирования C++. Специальное издание. Пер. с англ. – М.: Издательство Бином, 2017 г. – 1136 с.

Сайт cppreference: <https://en.cppreference.com/>

Приложение